

You don't know Bash: An introduction to Bash arrays

Enter the weird, wondrous world of Bash arrays.

31 May 2018 | [Robert Aboukhalil](#) | 432 | 11 comments



Image by : WOCinTech Chat. Modified by Opensource.com. CC BY-SA 4.0

Although software engineers regularly use the command line for many aspects of development, arrays are likely one of the more obscure features of the command line (although not as obscure as the regex operator `=~`). But obscurity and questionable syntax aside, [Bash](#) arrays can be very powerful.

Wait, but why?

Writing about Bash is challenging because it's remarkably easy for an article to devolve into a manual that focuses on syntax oddities. Rest assured, however, the intent of this article is to avoid having you RTFM.

A real (actually useful) example

To that end, let's consider a real-world scenario and how Bash can help: You are leading a new effort at your company to evaluate and optimize the runtime of your internal data pipeline. As a first step, you want to do a parameter sweep to evaluate how well the pipeline makes use of threads. For the sake of simplicity, we'll treat the pipeline as a compiled C++ black box where the only parameter we can tweak is the number of threads reserved for data processing: `./pipeline --threads 4`.

The basics

The first thing we'll do is define an array containing the values of the `--threads` parameter that we want to test:

```
allThreads=(1 2 4 8 16 32 64 128)
```

In this example, all the elements are numbers, but it need not be the case—arrays in Bash can contain both numbers and strings, e.g., `myArray=(1 2 "three" 4 "five")` is a valid expression. And just as with any other Bash variable, make sure to leave no spaces around the equal sign. Otherwise, Bash will treat the variable name as a program to execute, and the `=` as its first parameter!

Now that we've initialized the array, let's retrieve a few of its elements. You'll notice that simply doing `echo $allThreads` will output only the first element.

To understand why that is, let's take a step back and revisit how we usually output variables in Bash. Consider the following scenario:

```
type="article"
```

Programming and development

- [Red Hat Developers Blog](#)
- [Programming cheat sheets](#)
- [Try for free: Red Hat Learning Subscription](#)
- [eBook: An introduction to programming with Bash](#)
- [Downloadable guide: 7 essential PyPI libraries and how to use them](#)
- [What is an IDE?](#)

```
echo "Found 42 $type"
```

Say the variable `$type` is given to us as a singular noun and we want to add an `s` at the end of our sentence. We can't simply add an `s` to `$type` since that would turn it into a different variable, `$types`. And although we could utilize code contortions such as `echo "Found 42 "$type"s"`, the best way to solve this problem is to use curly braces: `echo "Found 42 ${type}s"`, which allows us to tell Bash where the name of a variable starts and ends (interestingly, this is the same syntax used in JavaScript/ES6 to inject variables and expressions in [template literals](#)).

So as it turns out, although Bash variables don't generally require curly brackets, they are required for arrays. In turn, this allows us to specify the index to access, e.g., `echo ${allThreads[1]}` returns the second element of the array. Not including brackets, e.g., `echo $allThreads[1]`, leads Bash to treat `[1]` as a string and output it as such.

Yes, Bash arrays have odd syntax, but at least they are zero-indexed, unlike some other languages (I'm looking at you, `R`).

Looping through arrays

Although in the examples above we used integer indices in our arrays, let's consider two occasions when that won't be the case: First, if we wanted the `$i`-th element of the array, where `$i` is a variable containing the index of interest, we can retrieve that element using: `echo ${allThreads[$i]}`. Second, to output all the elements of an array, we replace the numeric index with the `@` symbol (you can think of `@` as standing for `all`): `echo ${allThreads[@]}`.

Looping through array elements

With that in mind, let's loop through `$allThreads` and launch the pipeline for each value of `--threads`:

```
for t in ${allThreads[@]}; do
  ./pipeline --threads $t
done
```

Looping through array indices

Next, let's consider a slightly different approach. Rather than looping over array *elements*, we can loop over array *indices*:

```
for i in ${!allThreads[@]}; do
  ./pipeline --threads ${allThreads[$i]}
done
```

Let's break that down: As we saw above, `${allThreads[@]}` represents all the elements in our array. Adding an exclamation mark to make it `${!allThreads[@]}` will return the list of all array indices (in our case 0 to 7). In other words, the `for` loop is looping through all indices `$i` and reading the `$i`-th element from `$allThreads` to set the value of the `--threads` parameter.

This is much harsher on the eyes, so you may be wondering why I bother introducing it in the first place. That's because there are times where you need to know both the index and the value within a loop, e.g., if you want to ignore the first element of an array, using indices saves you from creating an additional variable that you then increment inside the loop.

Populating arrays

So far, we've been able to launch the pipeline for each `--threads` of interest. Now, let's assume the output to our pipeline is the runtime in seconds. We would like to capture that output at each iteration and save it in another array so we can do various manipulations with it at the end.

Some useful syntax

But before diving into the code, we need to introduce some more syntax. First, we need to be able to retrieve the output of a Bash command. To do so, use the following syntax: `output=$( ./my_script.sh )`, which will store the output of our commands into the variable `$output`.

The second bit of syntax we need is how to append the value we just retrieved to an array. The syntax to do that will look familiar:

```
myArray+=( "newElement1" "newElement2" )
```

The parameter sweep

Putting everything together, here is our script for launching our parameter sweep:

```
allThreads=(1 2 4 8 16 32 64 128)
allRuntimes=()
for t in ${allThreads[@]}; do
    runtime=$(./pipeline --threads $t)
    allRuntimes+=( $runtime )
done
```

And voilà!

What else you got?

In this article, we covered the scenario of using arrays for parameter sweeps. But I promise there are more reasons to use Bash arrays—here are two more examples.

Log alerting

In this scenario, your app is divided into modules, each with its own log file. We can write a cron job script to email the right person when there are signs of trouble in certain modules:

```
# List of logs and who should be notified of issues
logPaths=("api.log" "auth.log" "jenkins.log" "data.log")
logEmails=("jay@email" "emma@email" "jon@email" "sophia@email")

# Look for signs of trouble in each log
for i in ${!logPaths[@]}; do
    log=${logPaths[$i]}
    stakeholder=${logEmails[$i]}
    numErrors=$( tail -n 100 "$log" | grep "ERROR" | wc -l )

    # Warn stakeholders if recently saw > 5 errors
    if [[ "$numErrors" -gt 5 ]]; then
        emailRecipient="$stakeholder"
        emailSubject="WARNING: ${log} showing unusual levels of errors"
        emailBody="${numErrors} errors found in log ${log}"
        echo "$emailBody" | mailx -s "$emailSubject" "$emailRecipient"
    fi
done
```

API queries

Say you want to generate some analytics about which users comment the most on your Medium posts. Since we don't have direct database access, SQL is out of the question, but we can use APIs!

To avoid getting into a long discussion about API authentication and tokens, we'll instead use [JSONPlaceholder](#), a public-facing API testing service, as our endpoint. Once we query each post and retrieve the emails of everyone who commented, we can append those emails to our results array:

```
endpoint="https://jsonplaceholder.typicode.com/comments"
allEmails=()

# Query first 10 posts
for postId in {1..10}; do
    # Make API call to fetch emails of this post's commenters
    response=$(curl "${endpoint}?postId=${postId}")

    # Use jq to parse the JSON response into an array
    allEmails+=( $( jq '.[].email' <<< "$response" ) )
done
```

Note here that I'm using the [jq tool](#) to parse JSON from the command line. The syntax of `jq` is beyond the scope of this article, but I highly recommend you look into it.

As you might imagine, there are countless other scenarios in which using Bash arrays can help, and I hope the examples outlined in this article have given you some food for thought. If you have other examples to share from your own work, please leave a comment below.

But wait, there's more!

Since we covered quite a bit of array syntax in this article, here's a summary of what we covered, along with some more advanced tricks we did not cover:

Syntax	Result
<code>arr=()</code>	Create an empty array
<code>arr=(1 2 3)</code>	Initialize array
<code>\${arr[2]}</code>	Retrieve third element
<code>\${arr[@]}</code>	Retrieve all elements
<code>\${!arr[@]}</code>	Retrieve array indices
<code>\${#arr[@]}</code>	Calculate array size
<code>arr[0]=3</code>	Overwrite 1st element
<code>arr+=(4)</code>	Append value(s)
<code>str=\$(ls)</code>	Save <code>ls</code> output as a string
<code>arr=(\$(ls))</code>	Save <code>ls</code> output as an array of files
<code>\${arr[@]:s:n}</code>	Retrieve <code>n</code> elements starting at index <code>s</code>

One last thought

As we've discovered, Bash arrays sure have strange syntax, but I hope this article convinced you that they are extremely powerful. Once you get the hang of the syntax, you'll find yourself using Bash arrays quite often.

Bash or Python?

Which begs the question: *When should you use Bash arrays instead of other scripting languages such as Python?*

To me, it all boils down to dependencies—if you can solve the problem at hand using only calls to command-line tools, you might as well use Bash. But for times when your script is part of a larger Python project, you might as well use Python.

For example, we could have turned to Python to implement the parameter sweep, but we would have ended up just writing a wrapper around Bash:

```
import subprocess

all_threads = [1, 2, 4, 8, 16, 32, 64, 128]
all_runtimes = []

# Launch pipeline on each number of threads
for t in all_threads:
    cmd = './pipeline --threads {}'.format(t)

    # Use the subprocess module to fetch the return output
    p = subprocess.Popen(cmd, stdout=subprocess.PIPE, shell=True)
    output = p.communicate()[0]
    all_runtimes.append(output)
```

Since there's no getting around the command line in this example, using Bash directly is preferable.

Time for a shameless plug

This article is based on a talk I gave at [OSCON](#), where I presented the live-coding workshop [You Don't Know Bash](#). No slides, no clickers—just me and the audience typing away at the command line, exploring the wondrous world of Bash.

This article originally appeared on [Medium](#) and is republished with permission.



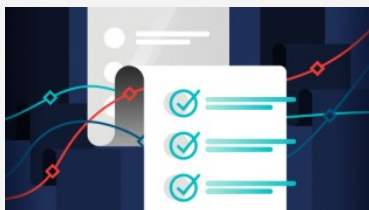


About the author

Robert Aboukhalil - Robert is a Bioinformatics Software Engineer at Invitae, which means that he spends his time... engineering software for bioinformatics purposes. Specifically, he develops cloud applications to enable the interactive analysis and exploration of genomics data. Robert has a Ph.D. in Bioinformatics from CSHL and a Bachelor in Computer Engineering from McGill.

[• More about me](#)

Recommended reading



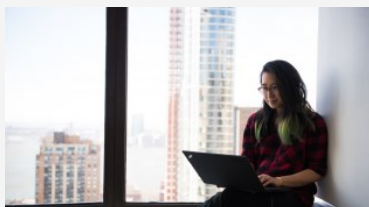
Perform unit tests using GoogleTest and CTest



Learn Rust in 2022



An open source developer's guide to 12-Factor App methodology



Set up a build system with CMake and VSCodium



5 ways to learn the C programming language in 2022



5 tips for learning a new programming language in 2022

11 Comments, [Register](#) or [Log in](#) to post a comment.



Roger on 01 Jun 2018

Fabulous!

Not only did you cover a subject or problem well, but you also provided real life working examples for explaining to readers where a particular function/feature is useful! This is a tactic often overlooked by most writers or speakers. Not only this, but you also provided the data lacking bias towards Python!



[Robert Aboukhalil](#) on 01 Jun 2018

Thanks Roger!



Greg M on 01 Jun 2018

Thanks for the great article!

One minor correction is needed. Your description of the behavior of `$(arr[@]:s:n)` as "Retrieve elements at indices n to s+n" should be "Retrieve elements at indices s to s+(n-1)". Or maybe more clearly, "Retrieve n elements beginning at index s"



[Robert Aboukhalil](#) on 01 Jun 2018

Good call, I'll make sure to fix that



Subrat on 01 Jun 2018

I like the way you have used the real world examples to make things more clear. Comparison with Python, similarity with JavaScript, are a few good ways you have used to attract programmers to try out some under-used things like bash arrays (particularly when bash is now available on almost

every os, including windows).



[Robert Aboukhalil](#) on 01 Jun 2018

Thanks Subrat!



[Ljubomir Ljubojevic](#) on 05 Jun 2018

I have been using BASH arrays for 9 years now, since 2009. I wanted CLI menu-based tool to manipulate/login my wireless routers and I used arrays to load router records from flat-text file (I am not concerned of security issues, it is stored in root account). It reached 2900 lines of code but is very functional and I can ssh and use menu-based app.



[Oleksii Tsvietnov](#) on 05 Jun 2018

There is another small bug in the text. Basically, the situation is similar to the difference between `$*` and `"$@"`. If you evaluate arrays to get all elements with `*` symbol, then it's ok to write `${array[*]}`. But in case of `@` symbol, it always has to be used with double quotes such as `"${array[@]}"`. It's not so critical until there are elements with space symbol, because then you'll lose a real value of a particular element.

Here is an example:

```
$ array1=(1 "two three" 4 five)
```

```
$ for i in ${array1[*]}; do echo ${i}; done
```

```
1
two
three
4
five
```

```
$ for i in ${array1[@]}; do echo ${i}; done
```

```
1
two
three
4
five
```

```
$ for i in "${array1[@]}"; do echo ${i}; done
```

```
1
two three
4
five
```



[balu](#) on 05 Jun 2018

nice yes.. pl mention this output too.. just for the sake of it:

```
for i in "${array1[*]}"; do echo ${i}; done
```

```
1 two three 4 five
```



[mabrown](#) on 09 Jun 2018

You use an example of `type="article"`.

Recommendation: `"help type"` in a bash shell and note that this is a shell builtin, so it would be better to name the shell variable containing the word `"article"` something else.



[Marty McGowan](#) on 10 Jun 2018

I've posted a review here:

<http://mcgowans.org/marty3/commonplace/software/arraysProCon.html>

where I come down on the side of functions. I've been too long function advocate to do otherwise, but I do encourage folks to take advantage of adding arrays to their practice, and `_then_` think about functions.

The article leads to my `"Commonplace"` book, largely devoted to the bash shell. Also, I've published `"Shell Functions"` on leanpub, but it's badly in need of revision.



Get the highlights in your inbox every week.

Find us:



[Privacy Policy](#) | [Terms of Use](#) | [Contact](#) | [Meet the Team](#) | [Visit opensource.org](#)

For more discussion on open source and the role of the CIO in the enterprise, join us at [The EnterprisersProject.com](#).

The opinions expressed on this website are those of each author, not of the author's employer or of Red Hat.

Opensource.com aspires to publish all content under a [Creative Commons license](#) but may not be able to do so in all cases. You are responsible for ensuring that you have the necessary permission to reuse any work on this site. Red Hat and the Red Hat logo are trademarks of Red Hat, Inc., registered in the United States and other countries.

Copyright ©2021 Red Hat, Inc.

open
source
.com